Exercise 2: Implementing Dependency Injection

BookRepository.java:

```
package com.exercise2.repository;


public class BookRepository {
    public void saveData() {
        System.out.println("[BookRepository] Book data saved securely via Setter Injection!");
    }
}
```

BookService.java:

```
package com.exercise2.service;


import com.exercise2.repository.BookRepository;


public class BookService {
    private BookRepository bookRepository;


    // Default No-Arg Constructor (Required for Setter Injection)
    public BookService() {
        System.out.println("[BookService] Initializing BookService instance...");
    }


    // Setter Method for Dependency Injection
    public void setBookRepository(BookRepository bookRepository) {
        System.out.println("[BookService] Injecting BookRepository dependency via setter method...");
        this.bookRepository = bookRepository;
    }
```

```
    public void saveBook() {
        if (bookRepository == null) {
            throw new IllegalStateException("BookRepository dependency has not been injected!");
        }
        System.out.println("[BookService] Business logic processing...");
        bookRepository.saveData();
    }
}
```

applicationContext.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository" class="com.exercise2.repository.BookRepository" />

    <bean id="bookService" class="com.exercise2.service.BookService">
        <property name="bookRepository" ref="bookRepository" />
    </bean>

</beans>
```

App.java:

```
package com.exercise2;

import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.exercise2.service.BookService;
```

```
public class App {

    public static void main(String[] args) {

        System.out.println("--- Starting Spring Container ---");


        // 1. Bootstrapping the container via XML configuration

        ClassPathXmlApplicationContext context = new
ClassPathXmlApplicationContext("applicationContext.xml");


        System.out.println("--- Container Initialized Successfully ---");


        // 2. Fetch the wired BookService bean

        BookService bookService = context.getBean("bookService", BookService.class);


        // 3. Test execution

        bookService.saveBook();


        // 4. Clean up resources

        context.close();

    }

}
```

Output: